

Video 2 — Demo Agent Skill

23127195 · Skill `eshop-automation` · mục tiêu 5–6 phút Quay face-cam trên máy tính.

Feature demo: FR-05 — Tìm kiếm sản phẩm (*feature MỚI, chưa từng tự động hoá*) Để nói skill phải *reusable on additional features*, nên dùng trên feature mới mới chứng minh được. Demo lại FR-09 thì người xem không phân biệt được skill có tác dụng gì hay chỉ chạy lại thứ có sẵn.

 **Điểm ăn tiền:** FR-05 render từ khoá tìm kiếm bằng `dangerouslySetInnerHTML` nên **có XSS thật**. Đã kiểm chứng: payload `<img src=x onerror=...>` **thực thi được**. Skill sẽ phát hiện lỗi này **ngay trên video** ở bước probe.

Chuẩn bị

```
cd D:\Kiem_thu\HW4\HW04-TESTING
node eshop-sut/backend/database.js
```

Hai terminal nền: backend + frontend-web.

- ☐ Chưa có file `tests/fr05-search.spec.js` (demo từ số 0 cho thật)
- ☐ Mở sẵn: **Claude Code trong VS Code · Terminal · Chrome**

1 · Mở đầu — 30 giây

Face-cam:

Em chào thầy cô. Em là **Trần Minh Hùng, MSSV 23127195**. Đây là video thứ hai của HW04, demo phần **Agent Skill**.

Video trước em trình bày bộ test cho ba feature. Video này em chứng minh em đã **đóng gói toàn bộ quy trình đó thành một skill tái sử dụng được**, và sẽ dùng nó ngay bây giờ cho **một feature hoàn toàn mới** — FR-05, tìm kiếm sản phẩm.

2 · Giới thiệu skill — 1 phút

VS Code → `.claude/skills/eshop-automation/SKILL.md`

Đây là file skill. Điểm em muốn nhấn mạnh: nó **không phải lý thuyết chép từ tài liệu**, mà là kết tinh từ những lỗi em đã thực sự mắc khi làm ba feature trước. Mỗi quy tắc ứng với một lần debug mất công.

(Cuộn bước 1–2)

Bước 1: đọc đặc tả trước khi đọc code. Vì đưa mã nguồn cho AI trước thì nó viết test khớp hành vi hiện tại — loại test đó luôn xanh và không bao giờ tìm ra lỗi, nó biến bug thành đặc tả.

Bước 2: probe DOM thật, tuyệt đối không đoán. Trong hệ thống này `getByLabel` không bao giờ hoạt động vì label không gắn `htmlFor`. Riêng lỗi này em mắc **hai lần** — một ở trang đăng ký, một ở trang admin.

(Cuộn bước 4 và 7)

Bước 4: không bao giờ chứng minh "không xảy ra" bằng cách kiểm tra URL chưa đổi — nó luôn khớp ngay lập tức.

Bước 7: con số không phải bằng chứng. Đã có lần năm test báo đỏ nhưng vì lý do hoàn toàn khác với lỗi chúng nhắm tới, trong khi tổng pass/fail trông vẫn y như kỳ vọng.

(Cuộn bảng SUT facts)

Cuối file là bảng sự thật về hệ thống em tốn công phát hiện: mật khẩu admin thật, cơ chế khoá tài khoản chỉ sau hai lần sai, Vite chỉ bind IPv6... Lần sau ai dùng skill này khỏi phải mò lại.

3 · Gọi skill — 30 giây

Claude Code — gõ:

Dùng skill eshop-automation để tự động hoá FR-05 – Product listing and search.
Làm đúng quy trình trong skill: probe DOM thật trước, dữ liệu để ngoài file,
assertion không race, rồi chạy 3 browser.

Em chỉ nêu **tên skill** và **tên feature**, không mô tả lại quy trình — vì toàn bộ hướng dẫn đã nằm trong file skill.

4 • ★ Bước probe phát hiện XSS — 1 phút

(Claude Code đang chạy script probe)

Đúng như skill quy định, việc đầu tiên nó làm **không phải viết test**, mà là dò xem trang tìm kiếm thực sự có gì và phản ứng thế nào với đầu vào đặc biệt.

(Kết quả probe hiện ra)

Và đây là lý do bước probe đáng giá. Từ khoá tìm kiếm được render bằng `dangerouslySetInnerHTML`, nên khi nhập một payload XSS thì **đoạn mã đó thực thi thật** — biến `window.__xss` được gán giá trị 1.

Đây là lỗ hổng **Cross-Site Scripting** thật sự. Nếu viết test ngay từ mô tả feature, em sẽ chỉ kiểm tra "tìm kiếm có ra kết quả không" và **bỏ sót hoàn toàn** lỗi này.

(Mở Chrome làm lại bằng tay để chứng minh)

5 • Sinh data file và spec — 45 giây

(Claude Code tạo `tests/data/fr05-*` và `tests/fr05-search.spec.js`)

Sau khi đã biết chắc hệ thống hoạt động thế nào, nó mới sinh test. Theo bước 3 của skill, toàn bộ dữ liệu ra file CSV/JSON riêng — để cấm viết cứng trong spec.

(Mở file spec)

Các assertion đều gắn nhãn P1 đến P6. Chú ý test XSS: nó **không kiểm tra qua URL** mà kiểm tra trực tiếp đoạn mã có thực thi hay không — đúng nguyên tắc "assertion không race" ở bước 4.

6 · Chạy 3 browser + report — 1 phút 30

```
node scripts/run-multibrowser.mjs tests/fr05-search.spec.js
```

Nói trong lúc chờ:

Bước 6 của skill: chạy trên cả ba engine, mỗi engine một báo cáo riêng.

Toàn bộ từ lúc em gõ một câu prompt tới khi có bộ test hoàn chỉnh chạy đa trình duyệt chỉ mất vài phút. Không có skill thì em phải mô tả lại từ đầu tất cả quy tắc kia — và nhiều khả năng lại mắc đúng lỗi cũ, vì **chính em đã mắc lỗi selector hai lần** trên hai màn hình khác nhau.

(Xong → mở report)

```
npx playwright show-report playwright-report/fr05-search-chromium
```

Báo cáo có đầy đủ **Run by 23127195** kèm timestamp, giống hệt ba feature trước — vì cấu hình đó đã nằm sẵn trong dự án và skill biết cách dùng.

7 · Kết — 30 giây

Face-cam:

Tóm lại: skill này biến quy trình em rút ra sau ba feature thành thứ **dùng lại được cho feature thứ tư** mà không phải nhớ lại gì. Nó không chỉ ghi *phải làm gì*, mà quan trọng hơn là ghi **vì sao** — mỗi quy tắc gắn với một lỗi thật em đã mắc.

Và như thầy cô vừa thấy, ngay lần dùng đầu tiên trên feature mới, nó đã giúp em phát hiện thêm **một lỗ hổng XSS**.

Em cảm ơn thầy cô đã theo dõi.

Kiểm tra sau khi quay

- ☐ Thấy rõ skill được **nạp và sử dụng** (không phải chỉ đọc file)
- ☐ Đủ chu trình: probe → data file → spec → 3 browser → report
- ☐ Có cảnh phát hiện **XSS**
- ☐ YouTube → **Unlisted** → dán link vào `README.md`
- ☐ Nếu FR-05 ra bug mới → **tạo Issue #16** cho lỗi XSS

Phương án dự phòng

Nếu lúc quay Claude Code trục trặc: quay lại quá trình dùng skill trên **FR-14** (feature đã làm), vừa chạy vừa đối chiếu từng bước với `SKILL.md`. Video vẫn hợp lệ vì đề chỉ yêu cầu "*shows how you used the skill on a complete feature*" — nhưng sức thuyết phục kém hơn hẳn so với làm trên feature mới.